TEAM BLUEPRINT · ADTC 2026 · REVISION 2

Ìtàn

An offline agricultural extension agent for smallholder farmers — updated to reflect what the corpus actually is, not what it was planned to be.

CHALLENGE	African Deep Tech Challenge (ADTC) 2026
DOMAIN	Agriculture — on-device language model inference
SUPERSEDES	*Itan_ADTC2026_Blueprint.pdf* (v1, this document does not repeat v1's rationale — read v1 first for the scoring-formula argument, tool architecture and eight-week plan, which are unchanged)
REASON FOR V2	The corpus that got built is a different shape than the one v1 specified. v2 re-grounds the plan in the actual ingested data — measured by inspecting `chunks.parquet`, `structured.db`, and this session's retrieval tests — rather than the ~10,000-chunk, six-balanced-table corpus v1 assumed.
HEADLINE CHANGE	The corpus's real strength is **pest & disease diagnosis** (Tier B) — deep, research-paper-sourced coverage. Its real weakness is **exact zone-specific facts** (Tier A: crop calendars, agrochemical registrations) — thin, sparse, and not yet trustworthy enough to serve uncritically. v2 repositions the product around the first and scopes down promises about the second.
STATUS	Living document — corpus and engine numbers below were measured directly from repo artifacts during a live debugging session, not projected

How to read this document. v2 does not re-derive the scoring-formula thesis, the tool architecture, or the multilingual design — those are unchanged from v1 and still the plan. v2 replaces v1's *projections* (§4, §9 of the original) with what got actually built, and adds the concrete engineering issues found while manually testing retrieval this week. Where a v1 claim still holds, it isn't repeated here.

1 What The Corpus Actually Is

v1 planned "~10,000 curated chunks" across six evenly-populated structured tables. What got built is larger, lumpier, and much more concentrated in one area than planned: **pest and disease content, pulled heavily from academic literature rather than extension bulletins, dominates everything else.**

72,452
CORPUS CHUNKS

1,529
UNIQUE SOURCES

1.1 What topic the corpus is actually about

Topic tag	Chunks	Share	Read
unclassified	16,139	22.3%	Topic classifier gap, not corpus content — see §5.2
market	11,214	15.5%	Price/value-chain content — not one of the blueprint's Tier A/B categories, currently unused
disease_management	**9,740**	**13.4%**	**16,427 chunks combined — the single largest well-classified category in the whole corpus, and larger than crop_calendar + spacing + agrochemical + variety combined by more than 30×**
pest_management	**6,687**	**9.2%**	
fertilisation	8,309	11.5%	Second-strongest category
soil_management	7,504	10.4%	
irrigation	4,048	5.6%	
variety_selection	3,183	4.4%	
harvest	2,545	3.5%	
planting	2,152	3.0%	Thin relative to how often Tier A planting-window questions will be asked
storage	931	1.3%	Thinnest category with real content

> **Reading this straight.** The corpus was not built evenly against the blueprint's six target tables. It was built by whatever was available to harvest, and what was available skewed hard toward pest/disease research literature. That's not a defect to apologize for — it's a genuine asset. **The positioning in §4 leans into it instead of pretending the corpus is the balanced extension-bulletin library v1 pictured.**

1.2 Crop coverage (all ten target crops present, unevenly)

Crop	Chunks	Crop	Chunks
Maize	13,537	Groundnut	3,853
Rice	7,664	Soybean	3,495
Cassava	5,895	Cowpea	3,305
Sorghum	4,884	Yam	2,848
Tomato	4,566	Pepper	1,585

33,849 of 72,452 chunks (47%) carry no crop tag at all — mostly crop-agnostic guidance (general pest-control principles, soil chemistry) rather than a tagging failure, per spot-checks, but not verified exhaustively.

1.3 The structured knowledge base — six tables, three real, one nearly empty

v1's Tier A design (§3.3 of v1) assumes exact facts come from clean SQL rows the model recites without alteration. That guarantee is only as good as the rows underneath it. Measured directly from `corpus/structured.db`:

Table	Rows	% flagged needs_review	Avg confidence	Verdict
pest	14,369	99.5%	0.399	VOLUME, UNVERIFIED
fertilizer_rate	3,025	95.4%	0.602	VOLUME, UNVERIFIED
spacing	425	33.2%	0.734	USABLE, THIN
variety	263	20.2%	0.735	USABLE, THIN
crop_calendar	60	11.7%	0.832	HIGHEST QUALITY, FAR TOO THIN
agrochemical	1	0%	0.750	EFFECTIVELY UNBUILT

> **The number that matters most in this document.** 99.5% of the `pest` table's 14,369 auto-extracted rows are flagged `needs_review`, with a mean confidence of 0.399. Serving these directly as Tier A "the model never sees a prompt asking for a value, so it cannot invent one" facts would replace one hallucination risk (the model guessing) with another (an unreviewed extraction pipeline guessing, presented with Tier A's full certainty). v1's Tier A design assumed a hand-curated table; what actually got built is a bulk auto-extraction with confidence scores attached but rarely acted on.

1.4 What "confidence" and "needs_review" actually measure

`corpus/05_structure_extract.py` scans every document for a regex match — capitalized word(s) next to a pest keyword ("...worm", "...blight", "...borer") — then looks in the next 800 characters for section labels (`Symptoms:`, `Cultural control:`, `Chemical control:`). The formula is `confidence = 0.3 + 0.15 × (labels found nearby) + 0.1 (if a crop was nearby)`. No LLM, no human — a label-proximity count. It never looks at whether the captured text is actually correct, complete, or even coherent. Consequence: the highest confidence any row in the pest table ever reaches is 0.85, and even the small number of rows that clear the 0.7 "trusted" threshold include entries like `pest_name = "to control both\nleaf spot"` or `"the spread\nof rust"` — a regex capturing running prose around a keyword, not a clean extraction, regardless of what its confidence score says.

The curated tables existed, but had the same blind spot — found and fixed 2026-08-17. `engine/tools/pest_lookup/build_db.py` already canonicalizes pest names and groups/dedupes the 14,369 raw rows down to one record per (pest, crop). What it didn't check: whether a group's merged record had any actual content. **140 of 192 curated records (73%) had a matched pest name and crop but symptoms, cultural_control, AND chemical_control all empty** — a phantom match: `pest_lookup()` would return it as an ordinary hit (`data_available=True` , non-trivial confidence), indistinguishable from a real match until every field was inspected. A confident empty answer is worse than an honest miss, since nothing signals the caller to fall back to Tier B retrieval instead. **Fixed:** `build_db.py` now drops a group entirely if all three content fields are empty after merging. `pest_lookup.db` rebuilt: 192 → 75 records, all 15 existing `eval/pest_lookup` test cases still pass.

The fix is necessary, not sufficient — a concrete example that still gets through. `crop=maize, pest_name=armyworm` (337 raw rows merged, confidence 0.7/"trusted") — exactly the record that would answer this session's flagship fall-armyworm-on-maize question — has *empty* symptoms and chemical_control, and its one populated field, cultural_control, reads: *"effects Toepfer & Fallet 0.0-4.0 leaf damage index Research assessing recent feeding damage, e.g. effects of plant protection prod- Vegetative Davis 0-9 whorl & furl damage scale"* — a fragment of a leaf-damage *rating-scale methodology* table from the correct source paper, not actual control advice. It has non-empty content, so the empty-record filter above doesn't (and shouldn't) remove it. Retrieval (§2) handles this exact question well — 0.92 hit-rate, correctly surfaces the real paper. The structured `pest_lookup` tool, for the identical question, would not. Closing this gap needs either better section-boundary extraction (the label-window regex is too easily misled by nearby tables/citations) or an actual relevance/quality check — neither is a quick fix, and both are follow-up work, not done here.

The same undocumented-implicit-curation pattern applies to `crop_calendar.db` (22 records from 60 raw rows) and should be assumed to carry a similar risk until checked the same way. `fertilizer_rate` has no curated table at all yet and is queried closer to raw — same class of risk, unaudited.

2 What Retrieval Actually Does

v1 positions retrieval as a solved hybrid dense+BM25 pipeline (§3.1, §3.4 of v1: "Hybrid over pure dense — rare agronomic proper nouns are poorly represented in small embedders"). Manually querying `engine.retrieval.RetrievalEngine` this week surfaced two bugs that mean that description hasn't matched the running code.

2.1 BM25 was silently dead on every query

`engine/retrieval.py`'s `bm25_search()` read the sparse index with `self.bm25_data.get("doc_ids", [])`. The index builder, `corpus/07_bm25.py`, pickles the field as `"chunk_ids"`, not `"doc_ids"`. The mismatched key meant `doc_ids` was always empty, which tripped a guard clause and made `bm25_search()` return `[]` on every call, silently. **Every query this system has ever answered in testing has been dense-only**, despite "hybrid" being the architecture's Pillar 1. Fixed (`engine/retrieval.py:142`).

2.2 54% of the corpus was unreachable by either search path — fixed

Artifact	Chunks indexed, before	Chunks indexed, after re-embed
`chunks.parquet`	72,452	72,452
`vectors.npy`	33,342	72,452
`bm25.pkl`	33,342	72,452

> **Fixed.** `corpus/06_embed.py` and `corpus/07_bm25.py` were re-run against the current `chunks.parquet`. Confirmed directly afterward: zero chunk IDs orphaned in either direction across all three artifacts — every chunk in the corpus now has both a dense vector and a BM25 entry.

2.3 The real lever was candidate-pool depth, not fusion weight

With full coverage restored, the natural next question was whether the dense/BM25 fusion weighting needed correcting — the earlier (stale-index) measurement showed hybrid scoring worse than dense-only, which read like a weighting problem. A weight sweep against the 50-question validation set (`corpus/tune_rrf.py`, checked into the repo) said otherwise:

Fusion weight (dense : BM25)	Hit Rate@5, full corpus
1.0 : 0.0 (dense-only)	0.78
0.0 : 1.0 (BM25-only)	0.88
1.0 : 0.5	0.90
1.0 : 1.0 (plain, unweighted — no change needed)	**0.92**
1.0 : 1.5 / 1.0 : 2.0	0.90

Equal weighting was already the best configuration — de-weighting BM25 (the fix the earlier, stale-index number seemed to call for) would have made things *worse* now that coverage is fixed. The actual gap was candidate-pool depth: `retrieve()` only pulled `top_k*2` (8) candidates per search path before fusion, so on

many questions the correct chunk was outside the pool RRF ever got to consider, regardless of how the fusion math weighted it. Sweeping pool depth at fixed 1:1 weighting:

Candidate pool (× top_k)	Hit Rate@5
×1 (5 candidates/side)	0.78
×2 (10 — the old default)	0.78
×4 (20)	0.88
×8 (40)	**0.92**
×10 / ×12	0.92 (no further gain)

> **Shipped:** `RRF_CANDIDATE_POOL_MULT = 8` in `engine/retrieval.py`. ×8 is the point of diminishing returns, not an arbitrary round number — ×10 and ×12 gave no further improvement in this sweep. Cost is negligible: both dense and BM25 search already score against the whole corpus; this only changes how many top-scored candidates get sliced off before fusion runs.

2.4 A second instance of the same class of bug, caught while verifying the fix

Re-running `corpus/08_validate.py` after the pool-depth fix initially still reported 0.78, not 0.92. Cause: the validation harness didn't call `RetrievalEngine.retrieve()` at all — it hand-recomposed `dense_search()` + `bm25_search()` + `rrf_fuse()` with its own hardcoded `TOP_K * 2` pool size, independent of whatever `retrieve()` did. This is the exact class of drift the file's own docstring says a previous rewrite was meant to eliminate ("one engine, one set of numbers, always in sync by construction") — it just resurfaced one layer down, in a parameter instead of a whole duplicated method. Fixed by having the harness call `engine.retrieve()` directly rather than re-deriving its internals. **Lesson for the rest of the codebase:** any place that reconstructs pieces of `RetrievalEngine`'s pipeline instead of calling `retrieve()` is a latent version of this same bug, waiting for the next tuning change to silently stop applying to it.

2.5 Retrieval gaps that are corpus-shape, not bugs

Gap	Measured (full corpus, tuned pool depth)
All six named agro-ecological zones	Still 0% hit rate@5, every zone — unchanged by either fix
Per-crop gaps	**Resolved** — every one of the ten target crops now ≥60%; cassava, the last remaining gap, went 50%→83.3%
Variety selection tier	60% — now the weakest tier (was previously masked by the coverage/pool bugs); worth attention next
Pest/disease diagnosis tier	95% — fertiliser_rate close behind at 93.3%, crop_management_storage at 100%

Zone-specific answers (§3.3 of v1's Tier A: "Maize window in Oyo") remain unsupportable — this was never an engine bug. The corpus doesn't carry consistent zone-tagged content, and `crop_calendar` has only 60 rows total, most with empty zone/state fields. Both engine fixes above left this gap exactly where it was, which is itself useful confirmation that it's a corpus-content gap and not a retrieval-tuning one, per §4.4.

2.6 A concrete worked example, kept for the record

Query: *"My maize shows: ragged holes in the leaves and sawdust-like frass packed in the whorl. What is the problem and how should I treat it?"* — gold answer: fall armyworm, emamectin benzoate / chlorantraniliprole.

Before either fix, the top-4 returned stem-borer content (a different pest, overlapping symptom vocabulary) and an off-topic maize-streak-virus chunk — despite the corpus holding 398 chunks that mention "fall armyworm" by name, including a chunk from a paper titled *"Streamlining leaf damage rating scales for the fall armyworm on maize,"* correctly tagged `crop=maize, topic=pest_management`. After the BM25 key fix alone, a chunk from that same paper became the #1 result. After the full re-embed and pool-depth fix, all four returned chunks are maize + pest_management/unclassified — a clean, focused result set with no off-topic content at all.

3 Everything Else, Status Unchanged From v1's "Known Issues"

Not re-measured this session; listed here so this document is a complete current-state snapshot rather than requiring a cross-reference back to v1 or README.md.

Item	Status
Model weights	**NOT DOWNLOADED** — `model/` has only a README; `download_model.sh` not yet run
llama.cpp / `llama-server`	**NOT PRESENT** — no binary in repo or on this machine; nothing requiring the LLM has run end-to-end
UI (Tauri + React, telemetry panel)	**NOT STARTED** — no `ui/` directory exists
Thermal governor	**NOT STARTED**
Lookup decoding + phrase bank	**NOT STARTED** — v1 calls this "the sharpest speed technique available," still unimplemented
LoRA fine-tune	**NOT STARTED** — explicitly cuttable per v1, lowest priority by design
Router (Tier A/B/C/D classifier)	**92.7% CROSS-VALIDATED ACCURACY**, 220 gold questions, 5-fold. Weakest tier: D (refusal) recall at 64.7% even with the best method — refusal calibration is the one part of the router still meaningfully under-performing
Tools (`agri_calc`)	**IMPLEMENTED**, 120 unit tests present matching v1's target exactly
Tools (`crop_calendar` , `pest_lookup`)	**IMPLEMENTED, THIN TEST COVERAGE** — 16 and 15 test cases respectively, well under the `agri_calc` bar
Language normalisation lexicon	**~70 TERMS CURRENTLY** vs v1's ~1,200-term target
Profiler compatibility run	One historical run on record (`eval/benchmarks/submission_profiler_output.json`): 8.73 TPS, 17.7s time-to-first-token, 1.72GB peak RSS. Predates the router/retrieval fixes in this document and can't be reproduced right now without model weights + llama.cpp — treat as stale, not current.

4 Repositioning — What v2 Actually Changes

v1's one-sentence pitch promised parity across pest ID, fertiliser dosing, planting calendars, spray dilution and margins. The corpus that got built doesn't support that evenly, and pretending otherwise in front of judges who will specifically probe zone-specific and product-specific claims (v1 §5.2's own worked example is exactly this kind of probe) is a worse strategy than being precise about where the depth actually is.

4.1 Revised one-sentence pitch

> *Ìtàn is an offline pest-and-disease diagnostic agent for smallholder maize, cassava, rice, sorghum and tomato farmers — grounded in 16,000+ chunks of primary agronomic literature, citing its sources, refusing when it isn't sure — with fertiliser dosing, seed-rate and margin calculation as deterministic Python tools layered on top.*

This is narrower than v1's pitch. It is also true, measurable, and defensible under judge questioning in a way the broader claim currently is not.

4.2 Tier A needs a confidence gate AND a content gate, documented as a design decision

Add to v1 §3.3: a Tier A query only resolves directly to SQL if the underlying row's `confidence` is above a threshold, *and* the fields it would actually present are non-empty. Confidence alone isn't enough — §1.4 showed a record can score above threshold ("trusted") while its retained field is a garbled fragment, and before the 2026-08-17 fix, 73% of `pest_lookup.db` had passed its confidence bar while carrying zero content in every field. The empty-content gate is now applied in `pest_lookup/build_db.py` — apply the equivalent to `crop_calendar.db` (unaudited, same risk assumed) and `fertilizer_rate` (no curated table yet at all), and route anything that fails either gate to Tier B retrieval + honest sourcing instead of presenting an unreviewed or empty extraction as a certain fact.

4.3 Lead the demo and the video with pest diagnosis, not fertiliser calculation

v1's two-minute video plan (§13.2) opens on a fertiliser question. Given where the corpus's actual strength sits, opening on a pest/disease diagnosis question — ideally the fall-armyworm case in §2.6, since it's a real, reproducible, verifiable example with a citation trail already confirmed — better demonstrates the system's genuine depth. Keep the fertiliser tool-calling example (v1's "0.6 hectares… urea" question) as the second beat, since that demonstrates the tool-calling pillar, which is a separate strength from retrieval depth.

4.4 Scope zone-specific and agrochemical-specific claims down until the data supports them

Do not claim zone-specific planting windows (v1's flagship Tier A example, "Maize window in Oyo") or registered-product PHI lookups (agrochemical table: 1 row) as core features in judge-facing material until `crop_calendar` and `agrochemical` get a real curation pass. Both are cuttable-if-thin per v1's own cut order (§10.4) — this is that principle being applied honestly rather than the promise being kept in the pitch while the data underneath quietly isn't there.

5 Status & Immediate Next Steps

5.1 Done since the numbers in §1–§2 were first measured

Item	Result
Re-embed full corpus (`06_embed.py` + `07_bm25.py`)	**DONE** — 100% coverage, 72,452/72,452 chunks in both indices
RRF fusion weight/pool-depth tuning	**DONE** — equal weighting confirmed optimal; the real fix was `RRF_CANDIDATE_POOL_MULT=8` (§2.3)
Re-run `corpus/08_validate.py`	**DONE** — **0.92 hit-rate@5**, corrected to call `retrieve()` directly (§2.4)
Empty-content gate for `pest_lookup.db`	**DONE** — 140/192 (73%) phantom-match records dropped; 75 remain, all 15 existing tests pass (§1.4)

Hit-rate@5 progression across this debugging session, all on the same 50-question set: `0.86 (dense-only, stale, mislabeled hybrid) → 0.60 (hybrid wired, still stale) → 0.78 (coverage fixed) → 0.92 (pool depth fixed)`. 0.92 is the number to cite going forward.

5.2 Still open

1. **Wire the language normalisation layer into retrieval** — `engine/agent.py` computes `norm_q` but passes the raw, un-normalised question into both the embedding call (in `server.py`) and `retrieval_engine.retrieve()` ; the lexicon currently has no effect on what gets searched.
2. **Investigate the variety-selection tier** — now the weakest at 60% hit-rate (§2.5), a gap that was previously masked by the coverage and pool-depth bugs rather than a new regression.
3. **Audit `crop_calendar.db` for the same phantom-match pattern** just fixed in `pest_lookup.db` (§1.4) — not yet checked, same risk assumed until verified.
4. **Apply an empty-content (and eventually a real quality) gate to `fertilizer_rate`** — it has no curated table at all yet and is queried closer to raw than either `pest_lookup` or `crop_calendar` (§1.3, §4.2).
5. **Close the garbled-but-present gap** demonstrated by the maize/armyworm example (§1.4) — the empty-content gate catches phantom matches but not populated fields containing mis-scoped table/citation fragments. Needs better section-boundary extraction or a real relevance check, not a quick fix.
6. **Audit for the same class of bug found in §2.4** — anywhere else in the codebase that reconstructs pieces of `RetrievalEngine` 's pipeline by hand instead of calling `retrieve()` is carrying the same silent-drift risk that hid the pool-depth fix from validation the first time it was re-run.
7. **Download model weights and build/obtain llama.cpp** — nothing past retrieval has been exercised end-to-end; this blocks router-to-answer testing, the profiler run, and the demo video.